

Operating Systems: Assignment-6

Name: Suraj Shah

Admission Number: I24AI012

Q 1. Write a C program to simulate the following preemptive CPU Scheduling algorithms to find turnaround time, waiting time, average turnaround time and average waiting time.

a) FCFS b) SRTF c) Priority Scheduling (Preemptive and Non preemptive) d) SJF

Take required inputs from user such as process id, burst time, arrival time.

Code:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

typedef struct {
    int pid;
    int at;    // Arrival Time
    int bt;    // Burst Time
    int ct;    // Completion Time
    int tat;   // Turnaround Time
    int wt;    // Waiting Time
    int priority;
    int remaining_bt;
} Process;

void calculateTimes(Process p[], int n) {
    float total_tat = 0, total_wt = 0;

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++) {
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
        total_tat += p[i].tat;
        total_wt += p[i].wt;

        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt,
```

```

        p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt/n);
}

void fcfs(Process p[], int n) {
    int time = 0;

    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(p[j].at > p[j+1].at) {
                Process temp = p[j];
                p[j] = p[j+1];
                p[j+1] = temp;
            }
        }
    }

    for(int i = 0; i < n; i++) {
        if(time < p[i].at)
            time = p[i].at;

        time += p[i].bt;
        p[i].ct = time;
    }

    calculateTimes(p, n);
}

void sjf(Process p[], int n) {
    int completed = 0, time = 0;
    int visited[MAX] = {0};

    while(completed < n) {
        int idx = -1;
        int min_bt = INT_MAX;

        for(int i = 0; i < n; i++) {
            if(p[i].at <= time && !visited[i] && p[i].bt < min_bt) {
                min_bt = p[i].bt;
            }
        }
    }
}

```

```

        idx = i;
    }
}

if(idx != -1) {
    time += p[idx].bt;
    p[idx].ct = time;
    visited[idx] = 1;
    completed++;
} else {
    time++;
}
}

calculateTimes(p, n);
}

void srtf(Process p[], int n) {
    int completed = 0, time = 0;

    for(int i = 0; i < n; i++)
        p[i].remaining_bt = p[i].bt;

    while(completed < n) {
        int idx = -1;
        int min_bt = INT_MAX;

        for(int i = 0; i < n; i++) {
            if(p[i].at <= time && p[i].remaining_bt > 0 &&
                p[i].remaining_bt < min_bt) {
                min_bt = p[i].remaining_bt;
                idx = i;
            }
        }

        if(idx != -1) {
            p[idx].remaining_bt--;
            time++;

            if(p[idx].remaining_bt == 0) {
                p[idx].ct = time;
                completed++;
            }
        }
    }
}

```

```

        }
    } else {
        time++;
    }
}

calculateTimes(p, n);
}

void priorityScheduling(Process p[], int n, int preemptive) {
    int completed = 0, time = 0;

    for(int i = 0; i < n; i++)
        p[i].remaining_bt = p[i].bt;

    while(completed < n) {
        int idx = -1;
        int highest = INT_MAX;

        for(int i = 0; i < n; i++) {
            if(p[i].at <= time && p[i].remaining_bt > 0 &&
                p[i].priority < highest) {
                highest = p[i].priority;
                idx = i;
            }
        }

        if(idx != -1) {
            if(preemptive) {
                p[idx].remaining_bt--;
                time++;

                if(p[idx].remaining_bt == 0) {
                    p[idx].ct = time;
                    completed++;
                }
            } else {
                time += p[idx].remaining_bt;
                p[idx].remaining_bt = 0;
                p[idx].ct = time;
                completed++;
            }
        }
    }
}

```

```

        } else {
            time++;
        }
    }

    calculateTimes(p, n);
}

int main() {
    Process p[MAX], temp[MAX];
    int n, choice;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i+1);
        printf("Process ID: ");
        scanf("%d", &p[i].pid);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Priority (lower number = higher priority): ");
        scanf("%d", &p[i].priority);
    }

    do {
        for(int i = 0; i < n; i++)
            temp[i] = p[i];

        printf("\nCPU Scheduling Algorithms");
        printf("\n1. FCFS");
        printf("\n2. SJF (Non-Preemptive)");
        printf("\n3. SRTF (Preemptive SJF)");
        printf("\n4. Priority (Non-Preemptive)");
        printf("\n5. Priority (Preemptive)");
        printf("\n6. Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);

        switch(choice) {

```

```

        case 1: fcfs(temp, n); break;
        case 2: sjf(temp, n); break;
        case 3: srtf(temp, n); break;
        case 4: priorityScheduling(temp, n, 0); break;
        case 5: priorityScheduling(temp, n, 1); break;
        case 6: break;
        default: printf("Invalid choice!\n");
    }

} while(choice != 6);

return 0;
}

```

Output:

```

ubuntu@ubuntu:~$ ./fspj
Enter number of processes: 4

Process 1
Process ID: 1
Arrival Time: 0
Burst Time: 7
Priority (lower number = higher priority): 2

Process 2
Process ID: 2
Arrival Time: 2
Burst Time: 4
Priority (lower number = higher priority): 1

Process 3
Process ID: 3
Arrival Time: 4
Burst Time: 1
Priority (lower number = higher priority): 3

Process 4
Process ID: 4
Arrival Time: 5
Burst Time: 4
Priority (lower number = higher priority): 2

```

CPU Scheduling Algorithms

1. FCFS
2. SJF (Non-Preemptive)
3. SRTF (Preemptive SJF)
4. Priority (Non-Preemptive)
5. Priority (Preemptive):
6. Exit

Enter choice: 1

PID	AT	BT	CT	TAT	WT
1	0	7	7	7	0
2	2	4	11	9	5
3	4	1	12	8	7
4	5	4	16	11	7

Average Turnaround Time = 8.75

Average Waiting Time = 4.75

CPU Scheduling Algorithms

1. FCFS
2. SJF (Non-Preemptive)
3. SRTF (Preemptive SJF)
4. Priority (Non-Preemptive)
5. Priority (Preemptive):
6. Exit

Enter choice: 2

PID	AT	BT	CT	TAT	WT
1	0	7	7	7	0
2	2	4	12	10	6
3	4	1	8	4	3
4	5	4	16	11	7

Average Turnaround Time = 8.00

Average Waiting Time = 4.00

CPU Scheduling Algorithms

1. FCFS
2. SJF (Non-Preemptive)
3. SRTF (Preemptive SJF)
4. Priority (Non-Preemptive)
5. Priority (Preemptive):
6. Exit

Enter choice: 3

PID	AT	BT	CT	TAT	WT
1	0	7	16	16	9
2	2	4	7	5	1
3	4	1	5	1	0
4	5	4	11	6	2

Average Turnaround Time = 7.00

Average Waiting Time = 3.00

CPU Scheduling Algorithms

1. FCFS
2. SJF (Non-Preemptive)
3. SRTF (Preemptive SJF)
4. Priority (Non-Preemptive)
5. Priority (Preemptive):
6. Exit

Enter choice: 4

PID	AT	BT	CT	TAT	WT
1	0	7	7	7	0
2	2	4	11	9	5
3	4	1	16	12	11
4	5	4	15	10	6

Average Turnaround Time = 9.50

Average Waiting Time = 5.50

Rhythmbox **ling Algorithms**

1. FCFS
2. SJF (Non-Preemptive)
3. SRTF (Preemptive SJF)
4. Priority (Non-Preemptive)
5. Priority (Preemptive):
6. Exit

Enter choice: 5

PID	AT	BT	CT	TAT	WT
1	0	7	11	11	4
2	2	4	6	4	0
3	4	1	16	12	11
4	5	4	15	10	6

Average Turnaround Time = 9.25

Average Waiting Time = 5.25

Q 2. Implementation of CPU Scheduling (Pre-emptive Processes): i) Round Robin ii) Longest job first and iii) Longest Remaining Time First

Code:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

typedef struct {
    int pid;
    int at;    // Arrival Time
    int bt;    // Burst Time
    int ct;    // Completion Time
    int tat;   // Turnaround Time
    int wt;    // Waiting Time
    int remaining_bt;
} Process;

void calculate(Process p[], int n) {
    float total_tat = 0, total_wt = 0;

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++) {
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;

        total_tat += p[i].tat;
        total_wt += p[i].wt;

        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt,
            p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt/n);
}

void roundRobin(Process p[], int n, int tq) {
    int time = 0, completed = 0;

    for(int i = 0; i < n; i++)
```

```

    p[i].remaining_bt = p[i].bt;

while(completed < n) {
    int done = 1;

    for(int i = 0; i < n; i++) {
        if(p[i].remaining_bt > 0 && p[i].at <= time) {
            done = 0;

            if(p[i].remaining_bt > tq) {
                time += tq;
                p[i].remaining_bt -= tq;
            } else {
                time += p[i].remaining_bt;
                p[i].remaining_bt = 0;
                p[i].ct = time;
                completed++;
            }
        }
    }

    if(done)
        time++;
}

calculate(p, n);
}

void ljf(Process p[], int n) {
    int completed = 0, time = 0;
    int visited[MAX] = {0};

    while(completed < n) {
        int idx = -1;
        int max_bt = -1;

        for(int i = 0; i < n; i++) {
            if(p[i].at <= time && !visited[i] && p[i].bt > max_bt) {
                max_bt = p[i].bt;
                idx = i;
            }
        }
    }
}

```

```

        if(idx != -1) {
            time += p[idx].bt;
            p[idx].ct = time;
            visited[idx] = 1;
            completed++;
        } else {
            time++;
        }
    }
}

calculate(p, n);
}

void lrtf(Process p[], int n) {
    int completed = 0, time = 0;

    for(int i = 0; i < n; i++)
        p[i].remaining_bt = p[i].bt;

    while(completed < n) {
        int idx = -1;
        int max_bt = -1;

        for(int i = 0; i < n; i++) {
            if(p[i].at <= time && p[i].remaining_bt > 0 &&
                p[i].remaining_bt > max_bt) {
                max_bt = p[i].remaining_bt;
                idx = i;
            }
        }

        if(idx != -1) {
            p[idx].remaining_bt--;
            time++;

            if(p[idx].remaining_bt == 0) {
                p[idx].ct = time;
                completed++;
            }
        } else {
            time++;
        }
    }
}

```

```

    }
}

calculate(p, n);
}

int main() {
    Process p[MAX], temp[MAX];
    int n, choice, tq;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i+1);
        printf("Process ID: ");
        scanf("%d", &p[i].pid);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
    }

    do {
        for(int i = 0; i < n; i++)
            temp[i] = p[i];

        printf("\n1. Round Robin");
        printf("\n2. Longest Job First (Non-Preemptive)");
        printf("\n3. Longest Remaining Time First (Preemptive)");
        printf("\n4. Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter Time Quantum: ");
                scanf("%d", &tq);
                roundRobin(temp, n, tq);
                break;
            case 2:
                ljf(temp, n);

```

```
        break;
    case 3:
        lrtf(temp, n);
        break;
    }

} while(choice != 4);

return 0;
}
```

Output:

```
ubuntu@ubuntu:~$ ./rlr
Enter number of processes: 4

Process 1
Process ID: 1
Arrival Time: 0
Burst Time: 8

Process 2
Process ID: 2
Arrival Time: 1
Burst Time: 4

Process 3
Process ID: 3
Arrival Time: 2
Burst Time: 9

Process 4
Process ID: 4
Arrival Time: 3
Burst Time: 5
```

1. Round Robin
2. Longest Job first (Non-Preemptive)
3. Longest Remaining Time First (Preemptive)
4. Exit

Enter choice: 1

Enter Time Quantum: 3

PID	AT	BT	CT	TAT	WT
1	0	8	23	23	15
2	1	4	16	15	11
3	2	9	26	24	15
4	3	5	21	18	13

Average Turnaround Time = 20.00

Average Waiting Time = 13.50

1. Round Robin
2. Longest Job first (Non-Preemptive)
3. Longest Remaining Time First (Preemptive)
4. Exit

Enter choice: 2

PID	AT	BT	CT	TAT	WT
1	0	8	8	8	0
2	1	4	26	25	21
3	2	9	17	15	6
4	3	5	22	19	14

Average Turnaround Time = 16.75

Average Waiting Time = 10.25

1. Round Robin
2. Longest Job first (Non-Preemptive)
3. Longest Remaining Time First (Preemptive)
4. Exit

Enter choice: 3

PID	AT	BT	CT	TAT	WT
1	0	8	23	23	15
2	1	4	24	23	19
3	2	9	25	23	14
4	3	5	26	23	18

Average Turnaround Time = 23.00

Average Waiting Time = 16.50

1. Round Robin
2. Longest Job first (Non-Preemptive)
3. Longest Remaining Time First (Preemptive)
4. Exit

Enter choice: 4